

Architettura dei calcolatori

Leonardo Donati

7 luglio 2026

Indice

1	Introduzione	4
1.1	Tipi di architetture	4
2	Definizione di Calcolatore Elettronico	4
2.1	Perchè sistema?	4
2.1.1	Come descrivere un sistema?	4
2.1.2	Descrizione funzionale	4
2.1.3	Descrizione strutturale	5
2.1.4	Analisi (Reverse Engineering)	5
2.1.5	Sintesi	5
2.2	Perchè programmabile?	5
2.2.1	Astrazione	5
2.2.2	Livelli di astrazione	5
2.2.3	Livelli Hardware	6
2.2.4	Livelli fra Hardware e Software	6
2.3	Perchè digitali?	6
2.3.1	Dati	6
2.3.2	Differenza fra dati e segnali	6
2.4	Perchè elaborazione dell'informazione?	7
2.4.1	Informazione	7
2.4.2	Autoinformazione	7
2.4.3	Entropia	7
2.5	Perchè digital purpose?	7
2.5.1	Generazione 0	7
2.5.2	Generazione 1	7
2.5.3	Generazione 3	8
2.5.4	Generazione 4	8
3	Prestazioni	9
3.1	Bottleneck	9
3.2	Tempo di CPU	9
3.3	Clock per instruction (CPI)	9
3.4	Speed Up	9
3.5	Legge di Amdhal	10
4	Benchmark	11
5	Reti logiche	12
5.1	Rete combinatorie VS reti sequenziali	12
5.2	Algebra di Boole	12
5.2.1	Tabella di verità	12
5.2.2	Funzioni logiche di una variabile	12
5.2.3	Funzioni logiche di due variabili	12
5.2.4	Proprietà di interconnessione	13
5.2.5	Proprietà di decomposizione	13
5.2.6	Altre proprietà	13
5.2.7	Mappe di Karnaough	13
5.2.8	Letterale	13

5.2.9	Implicante	14
5.2.10	Mintermine	14
5.2.11	Maxtermine	14
5.2.12	Sintesi canonica	14
5.3	Multiplexer	15
5.4	Demultiplexer	15
5.5	Esercizio struttura calcolatore	16
5.6	Reti sequenziali	17
5.6.1	Alee	17
5.6.2	Elementi di Memoria	18
5.6.3	Set Reset	18
5.6.4	Latch e D-Latch	18
5.6.5	Flip Flop	18
5.7	Registri	18
5.7.1	Lettura	18
5.7.2	Scrittura	18
6	ISA (Instruction Set Architecture)	19
6.1	RISC-V Instruction Set	19
6.1.1	Operazioni aritmetiche	19
6.1.2	Registri nel RISC-V	19
6.1.3	Codifica delle istruzioni	20
6.1.4	Salti condizionati	20
6.1.5	Procedure	20
6.1.6	Stack e Heap	20
7	Architettura della CPU	21
7.1	Alee	22
7.1.1	Alee strutturali	22
7.1.2	Alee di dato	22
7.1.3	Alee di controllo	22
8	Memorie	23
8.1	Capacità	23
8.2	Caratteristiche fisiche	23
8.3	Modalità di accesso	23
8.3.1	Memorie RAM	24
8.3.2	Memorie ROM	24
8.4	Organizzazione della memoria	24
8.4.1	Organizzazione gerarchica	24
8.4.2	Memorie Cache	25
8.4.3	Prestazioni nell'organizzazione gerarchica	26
8.5	Miss	27
8.5.1	Esercizio:	27
8.6	Scrittura	28
8.7	Memoria virtuale	28
8.7.1	Page fault	28
8.7.2	Address mapping	29
8.7.3	TLB (Translation Lookaside Buffer)	29
9	Codici di Controllo	30
9.1	Distanza di Hamming	30
9.2	Codice di parità	31
9.3	Codice di Hamming	31
9.3.1	Esempio: Esercizio con m=4	31

10 Parallelismo	32
10.1 Architetture MIMD	33
10.1.1 UMA e NUMA	34
10.2 Architettura Multiple-Issue	34
10.3 Architettura Superpipeline	34
10.4 Architettura VLIW (Very Long Instruction Word)	34
10.5 Architettura Superscalare	34
10.6 Esecuzione Speculativa	34
10.6.1 RAT (Register Alias Table)	34
10.6.2 Fuori Ordine	34
10.7 Interrupt e Polling	35
10.7.1 Interruzioni	35
10.7.2 Faults	35
10.7.3 Traps	35
10.7.4 Aborts	35

1 Introduzione

Perchè è utile conoscere questa materia?

- **Conoscere** il funzionamento
- **Ottimizzare** le prestazioni
- **Gestire** organizzazione che richiedono un sistema di calcolo
- **Progettare** architetture dei calcolatori delle prossime generazioni

1.1 Tipi di architetture

Ci sono più tipi di architetture:

- **General purpose:** ha almeno una CPU, un sistema operativo e gli applicativi sono scritti da terzi. Per esempio un computer normale.
- **Embedded:** può non avere un sistema operativo, hardware e software è tutto progettato.
- **Domain specific:** con specifiche ottimizzate per alcuni domini applicativi
- **High performance** progettate solo per applicazioni ad elevate prestazioni

2 Definizione di Calcolatore Elettronico

I calcolatori elettronici o computer si possono definire **sistemi digitali di elaborazione dell'informazione, programmabili e general-purpose.**

2.1 Perchè sistema?

Nell'ambito scientifico, un sistema è un qualsiasi oggetto di studio che, pur essendo costituito da diversi elementi reciprocamente interconnessi e interagenti tra loro o con l'ambiente esterno, reagisce o evolve come un elemento unitario, con proprie leggi generali.

2.1.1 Come descrivere un sistema?

Un sistema viene descritto attraverso il suo modello in uno specifico dominio di rappresentazione che indica l'insieme di particolari considerati di interesse. Per ogni livello si può fornire:

- **Descrizione funzionale:** descrive il funzionamento del sistema in termini di suoi ingressi e delle sue uscite.
- **Descrizione strutturale:** descrive la struttura in termini di elementi primitivi che lo compongono.
- **Descrizione fisica:** descrive la effettiva realizzazione del sistema indicando in dettaglio i componenti impiegati e le loro interconnessioni.

2.1.2 Descrizione funzionale

Un sistema digitale che riceve in ingresso dati ed istruzioni e fornisce in uscita dati di risultati, ha, quindi, diversi compiti:

- **Gestione delle istruzioni:** deve memorizzare, recuperare, interpretare ed eseguire le istruzioni di un programma per lavorare sui dati e controllare tutto il sistema.
- **Elaborazione dei dati:** deve elaborare dati binari.
- **Memorizzazione dei dati:** i dati prima e dopo della elaborazione devono essere memorizzati in modo temporaneo ed in modo permanente.
- **Trasferimento dei dati:** i dati devono essere trasferiti in ingresso ed uscita a sistemi direttamente collegati o deve essere instaurata una comunicazione con dispositivi remoti.

2.1.3 Descrizione strutturale

Descrive la parte hardware composta dai dispositivi fisici ed in particolare da una o più CPU (Central Processing Unit), da memorie e dispositivi di I/O collegati da uno o più bus e la parte software, costituita da software di base e software applicativo, ossia dai programmi che codificano in linguaggio macchina uno specifico algoritmo.

2.1.4 Analisi (Reverse Engineering)

L'analisi è lo studio del funzionamento di un sistema noto a partire dalla sua struttura fino ad arrivare ad una sua descrizione comportamentale (o funzionale).

2.1.5 Sintesi

La sintesi di un sistema è al contrario il processo che porta alla progettazione (e in seguito alla realizzazione) della struttura del sistema, dei suoi componenti e delle connessioni tra componenti a partire dalla descrizione comportamentale fino alla descrizione strutturale.

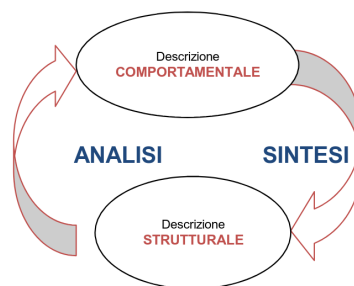


Figura 1: Schema analisi e sintesi.

2.2 Perché programmabile?

Un sistema che deve essere progettato o compreso viene definito ad un dato livello di astrazione. Il livello di astrazione dell'architettura dei calcolatori è quello del programmatore di basso livello (o meglio del compilatore) e quindi l'architettura del calcolatore consiste nella struttura e l'organizzazione hardware delle parti e l'Instruction Set Architecture (ISA) ossia l'insieme delle istruzioni, della struttura dei dati e dei meccanismi di controllo dei componenti.

2.2.1 Astrazione

Definire tutti i dettagli utili per comprenderne in dettaglio funzionamento o struttura ad uno specifico livello ignorando livelli di maggiore o minore dettaglio, e definendo interfacce per passare ai livelli maggiori o minori di astrazione.

2.2.2 Livelli di astrazione

Il calcolatore è molto complesso, quindi viene studiato in diversi domini di rappresentazione e a diversi livelli di astrazione. Per ogni progetto bisogna definire un modello, ossia astrazioni della realtà in cui si evidenziano aspetti di interesse e si sopprimono i dettagli non interessanti per il livello di astrazione in cui ci si collocano.

Il livello di astrazione è quindi il grado di dettaglio del modello e la rappresentazione di quali sono i metodi per tradurre i dettagli in un livello di astrazione precedente o successivo.

2.2.3 Livelli Hardware

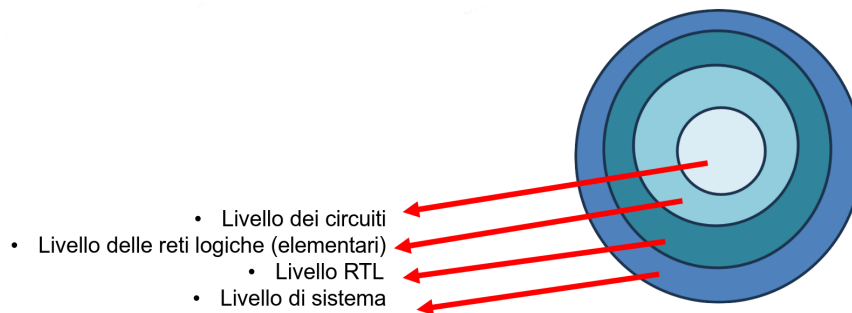


Figura 2: Livelli hardware di un calcolatore.

1. Livello dei circuiti.

2. **Livello delle reti logiche:** il cui linguaggio di descrizione è dato dal formalismo dell'algebra di Boole e degli automi a stati finiti.

3. **Livello RTL(Registry transfer level):** i componenti che si descrivono si definiscono a livelli non di bit ma di parole, si usa un linguaggio formale RTL o linguaggi di descrizione dell'hardware (HDL Hardware description Languages).

4. **Livello di sistema:** in cui si definiscono i componenti e le connessioni. Il linguaggio di descrizione è il linguaggio macchina, l'assembly e i linguaggi di programmazione come il C.

I linguaggi di descrizione dell'hardware più usati sono il VHDL (VHSIC Hardware Description Language, dove VHSIC è la sigla di Very High Speed Integrated Circuits), oppure il SystemC.

2.2.4 Livelli fra Hardware e Software

Le funzioni che si possono ottenere con una sintesi hardware sono le stesse ottenibili in software e viceversa, con un diverso rapporto tra prestazioni e flessibilità. Per questo hanno in comune alcuni livelli:

- **Livello di microarchitettura:** è il livello che descritto come RTL o come sistema definisce le scelte sui blocchi logici che implementano la CPU e l'interno dei componenti del calcolatori.
- **Livello dell'ISA (Instruction Set Architecture):** Livello che definisce l'architettura della CPU e di conseguenza ciò che il calcolatore può svolgere. Ogni produttore ha un manuale della CPU e del proprio linguaggio Macchina implementato nell'ISA.

2.3 Perché digitali?

Ogni calcolatore ha bisogno di dati digitali per compiere calcoli e processi.

2.3.1 Dati

I dati per un calcolatore sono tutte le grandezze quantificabili che vengono impiegate per un calcolo o per un processo. Alcuni esempi possono essere numeri, lettere, valori grafici, audio e video.

2.3.2 Differenza fra dati e segnali

I segnali sono una qualsiasi grandezza fisica che varia nel tempo e che può assumere un insieme di valori misurabili che possono essere trasmessi, memorizzati ed impiegati come dati in un computer.

Possono essere:

- **Analogici:** se possono assumere infiniti valori all'interno di un dato intervallo.
- **Digitali:** se possono assumere un numero finito, discreto, di valori sempre all'interno di un dato intervallo di variabilità e quindi rappresentabile con numeri.

I segnali nascono normalmente nella natura di tipo analogico e sono acquisiti da sensori e trasformati in digitale attraverso convertitori.

2.4 Perchè elaborazione dell'informazione?

L'informazione, indipendentemente dal suo significato, può essere creata, ossia codificata, trasmessa ed elaborata in modo misurabile. Il calcolatore elettronico ha il compito di elaborare l'informazione digitale.

2.4.1 Informazione

Si chiamano informazioni i dati che assumono un significato univoco, utile per una specifica applicazione. L'informazione è la riduzione di incertezza che si può avere a priori sul simbolo trasmesso.

2.4.2 Autoinformazione

Ogni simbolo, di un alfabeto $X = \{x_i, i = 1 \dots M\}$, x_i avente probabilità $P(x_i)$ di essere trasmesso ha una informazione data dalla quantità, non negativa, definita come la riduzione di incertezza che si può stimare a priori sul simbolo trasmesso:

$$I(x_i) = \log_B \frac{1}{P(x_i)} = -\log_B P(x_i)$$

2.4.3 Entropia

L'entropia è il contenuto informativo medio della sorgente o la misura della perdita di incertezza derivata, se si considera il valor medio, su tutto l'alfabeto dei simboli possibili, di $I(x_i)$:

$$H(x) = E[I(x_i)] = \sum_{i=1 \dots M} P(x_i) I(x_i)$$

NB: Dati M simboli: $0 < H(x) < \log M$

2.5 Perchè digital purpose?

I sistemi di elaborazione digital purpose hanno sempre determinate caratteristiche:

- Hanno sempre almeno una CPU.
- Dotato di sistema operativo, software realizzato da terze parti.
- La sua funzionalità è specificata dal software applicativo.
- Può avere unità di elaborazione aggiuntive, come GPU, TPU, ecc...

Ci sono state diverse generazioni di calcolatori elettronici, in ognuna delle quali ci sono state evoluzioni e invenzioni importanti.

2.5.1 Generazione 0

Generazione 0 dei computer, il calcolatore era ancora un oggetto studiato tendenzialmente per risolvere calcoli matematici con elementi meccanici o elettromeccanici.

2.5.2 Generazione 1

La Generazione 1 dei computer però nasce con la nascita dell'elettronica e dei tubi a vuoto i precursori dei transistor, per creare la macchina ENIGMA per codificare i messaggi tedeschi e la macchina COLOSSUS parzialmente progettata da Alan Turing per decodificarli. Il modello attuale dei calcolatori deriva dal modello definito da Von Neumann negli anni '50.

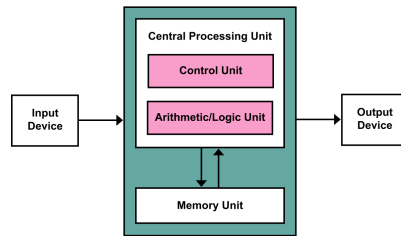


Figura 3: Architettura di Von Neumann.

2.5.3 Generazione 3

Detta generazione dei circuiti integrati ('65-'70-'80), l'aspetto caratterizzante è la integrazione elettronica dei computer che da essere grandi come stanze diventano di dimensioni più vicine a quelle dei calcolatori attuali.

2.5.4 Generazione 4

Nella 4° generazione o generazione dei VLSI ('80-), la larga scala di integrazione apre diverse possibilità a nuove forme di computers. Si sviluppano sempre più minicomputer accessibili anche a piccoli centri di calcolo che in assenza di internet hanno comunque accesso a dati e elaborazioni locali e possono collegarsi a reti mondiali sperimentali.

3 Prestazioni

Conoscere come calcolare le prestazioni di tutto il calcolatore o della CPU e dei vari componenti è molto importante sia in fase di progettazione di un nuovo prodotto sia in fase di analisi e per la scelta, l'acquisto e l'uso di un calcolatore. In particolare, si parla di:

- **Efficacia:** essere in grado di compiere il compito previsto.
- **Efficienza:** la possibilità di svolgere il compito con minori risorse possibili.

Nei calcolatori, le prestazioni dipendono molto dalla velocità, per questo è necessario analizzare diversi fattori:

- **Tempo di esecuzione (T_{exec}):** tempo di esecuzione per un singolo task.
- **Throughput:** quantità di task nell'unità di tempo.

3.1 Bottleneck

- **CPU-Bound** quelle applicazioni o programmi che sono limitate dalle prestazioni della CPU, tendenzialmente programmi di calcolo, di grafica, dove devono essere fatte molte operazioni di ALU e di memoria.
- **IO-Bound** quelle applicazioni o programmi che sono limitate dalle prestazioni delle periferiche di I/O, dai dischi anche distribuiti, dalla rete e dall'uomo; come ad esempio l'accesso ai Database o ai web server.
- **Memory-Bound** quelle applicazioni o programmi che sono limitate dalle prestazioni della memoria centrale come in Database in-memory e come per l'addestramento di reti neurali di grandi dimensioni. Spesso si migliora con una architettura efficace delle cache.

3.2 Tempo di CPU

Il calcolo delle prestazioni con un modello analitico semplice si interessa solo della CPU:

$$T_{cpu} = NccT_{clk} = \frac{Ncc}{f}$$

Dove:

- Ncc è il numero di cicli di clock.
- f è la frequenza di clock.
- T_{clk} è il tempo di clock.

3.3 Clock per instruction (CPI)

Numero di periodi di clock impiegati mediamente per eseguire una istruzione considerando un insieme di programmi, su una specifica CPU:

$$CPI = \frac{Ncc}{Ni} = \sum (x_i CPI_i)$$

Quindi, possiamo calcolare il tempo di CPU anche come:

$$T_{cpu} = NiCPI T_{clk} = \frac{NiCPI}{f}$$

Dove:

- Ni è il numero di istruzioni.

3.4 Speed Up

Speedup o accelerazione è un indice di miglioramento di quanto è migliorata la prestazione globale avendo migliorato una parte del sistema. È il rapporto tra la prestazione ottenuta con il miglioramento rispetto a quella ottenuta senza miglioramento.

$$SP = \frac{T_{exec-old}}{T_{exec-new}}$$

3.5 Legge di Amdhal

Il miglioramento delle prestazioni dovute ad un miglioramento di esecuzione e' limitato dalla frazione di tempo in cui tale miglioramento puo' essere applicato.

$$SP(overall) = \frac{1}{(1 - F_{enh}) + \frac{F_{enh}}{S_{enh}}} = \frac{1}{[1 - \sum_i (F_{enh_i})] + \sum_i (\frac{F_{enh_i}}{S_{enh_i}})}$$

Dove:

- F_{enh} (**fraction enhanced**) è la percentuale di tempo in cui ha effetto il miglioramento.
- S_{enh} (**speedup enhanced**) è il valore di tale miglioramento.

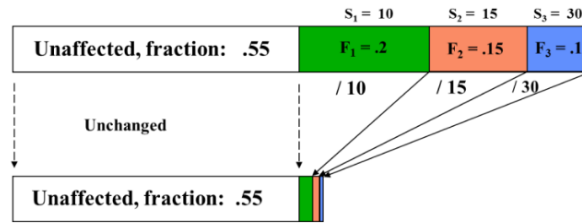


Figura 4: Speed up effittivo di tre CPU.

4 Benchmark

5 Reti logiche

Il livello delle reti logiche è il livello di astrazione che studia i sistemi digitali a livello di componenti logici elementari, chiamati gate, indipendentemente dalla loro realizzazione fisica. Invece, dal punto di vista funzionale, la rete logica è la astrazione di un sistema digitale avente n segnali binari di ingresso ed m segnali binari di uscita ed una funzione logica che mette in relazione i segnali di uscita con quelli di ingresso.

5.1 Rete combinatorie VS reti sequenziali

Una rete è combinatoria se ogni segnale di uscita dipende solo dai valori degli ingressi in quell'istante, invece, una rete è sequenziale se ogni segnale di uscita dipende dai valori degli ingressi in quell'istante e dai valori che gli ingressi hanno assunto negli istanti precedenti.

5.2 Algebra di Boole

L'algebra di Boole è un sistema matematico che descrive funzioni di variabili binarie, composto da:

- Un insieme di simboli $B = \{0, 1\}$.
- Un insieme di operazioni $O = \{+, \cdot, '\}$.
 1. Somma logica $+$.
 2. Prodotto logico $\cdot$.
 3. Complementazione $'$.
- Un insieme di postulati(assiomi) P .

5.2.1 Tabella di verità

La Tabella di verità è una: tabella che associa tutte le possibili combinazioni degli ingressi alle corrispondenti configurazione delle uscite, e indica esaurientemente il comportamento della rete logica.

x_1	$\dots$	x_n	y_1	$\dots$	y_m
1	$\dots$	0	1	$\dots$	-
0	$\dots$	1	-	$\dots$	1
1	$\dots$	0	1	$\dots$	0
0	$\dots$	1	0	$\dots$	1
1	$\dots$	0	1	$\dots$	-

Una tabella di verità di n variabili di ingresso ha 2^n righe. Si possono creare 2^{2^n} tabelle delle uscite.

5.2.2 Funzioni logiche di una variabile

Sono 4 possibili funzioni:

- **Massa:** l'uscita vale sempre 0.
- **Buffer:** l'uscita è sempre uguale all'ingresso.
- **NOT o complementazione:** l'uscita è sempre l'opposto dell'ingresso.
- **Vcc:** l'uscita vale sempre 1.

5.2.3 Funzioni logiche di due variabili

Sono 16 possibili funzioni:

- **AND:** l'uscita vale 1 se entrambi gli ingressi sono 1.
- **OR:** l'uscita vale 1 se almeno uno dei due ingressi vale 1.
- **XOR** l'uscita vale 1 se almeno uno dei due ingressi vale 1 ma non entrambi.

- **NOR** l'uscita vale 1 se entrambi gli ingressi sono 0.
- **XAND** l'uscita vale 1 se entrambi gli ingressi sono diversi da 1.
- **XNOR** l'uscita vale 1 se gli ingressi sono uguali.

5.2.4 Proprietà di interconnessione

L'interconnessione di più reti logiche, aventi per ingresso segnali esterni o uscite di altre reti logiche e per uscite segnali di uscita esterne o ingressi di altre reti logiche, è ancora una rete logica.

5.2.5 Proprietà di decomposizione

Una rete logica complessa può essere decomposta in reti logiche più semplici.

5.2.6 Altre proprietà

Identità	(T1) $X + 0 = X$	Associativa	(T7) $(X + Y) + Z = X + (Y + Z) = X + Y + Z$
	(T1') $X \cdot 1 = X$		(T7') $(X \cdot Y) \cdot Z = X \cdot (Y \cdot Z) = X \cdot Y \cdot Z$
Elementi nulli	(T2) $X + 1 = 1$	Assorbimento	(T8) $X + X \cdot Y = X$
	(T2') $X \cdot 0 = 0$		(T8') $X \cdot (X + Y) = X$
Idempotenza	(T3) $X + X = X$	Distribuzione	(T9) $X \cdot Y + X \cdot Z = X \cdot (Y + Z)$
	(T3') $X \cdot X = X$		(T9') $(X + Y) \cdot (X + Z) = X + Y \cdot Z$
Involuzione	(T4) $(X')' = X$	Combinazione	(T10) $(X + Y) \cdot (X' + Y) = Y$
Complementarietà	(T5) $X + X' = 1$		(T10') $X \cdot Y + X' \cdot Y = Y$
	(T5') $X \cdot X' = 0$	Consenso	(T11) $(X + Y) \cdot (X' + Z) \cdot (Y + Z) = (X + Y) \cdot (X' + Z)$
Commutativa	(T6) $X + Y = Y + X$		(T11') $X \cdot Y + X' \cdot Z + Y \cdot Z = X \cdot Y + X' \cdot Z$
	(T6') $X \cdot Y = Y \cdot X$	De Morgan	(T12) $(X + Y)' = X' \cdot Y'$
			(T12') $(X \cdot Y)' = X' + Y'$

Figura 5: Altre proprietà algebra lineare.

5.2.7 Mappe di Karnaugh

Deve rispettare la distanza di Hamming, ossia le configurazioni di bit adiacenti non devono differire per più di un bit.

		x_2, x_1			
		00	01	11	10
x_0	0	0	0	-	1
	1	1	1	1	1

Figura 6: Esempio di mappa di Karnaugh.

5.2.8 Letterale

Sia dato un circuito combinatorio con n variabili di ingresso, $X: \{X_1 \dots X_n\}$. Si chiama letterale ogni occorrenza di una singola variabile, sia in forma semplice X_i che complementata X'_i . Se abbiamo $Z = ABC + AB'$, allora A, B, B' e C sono letterali della funzione.

5.2.9 Implicante

Si dice implicante di una funzione $f(X_1, \dots, X_n)$ un prodotto $P(X')$ con $X' \subseteq X$ di letterali tale che $P = 1 \implies f = 1$. Si chiama implicante primo di una funzione f un implicante di f che non è coperto da un altro implicante di f con meno letterali. Gli implicanti e gli implicanti primi si possono vedere nelle mappe di Karnaugh dove i raggruppamenti rettangolari che coprono le uscite pari a 1, indicano gli implicanti e gli implicanti primi se non esistono altri raggruppamenti rettangolari che li coprono.

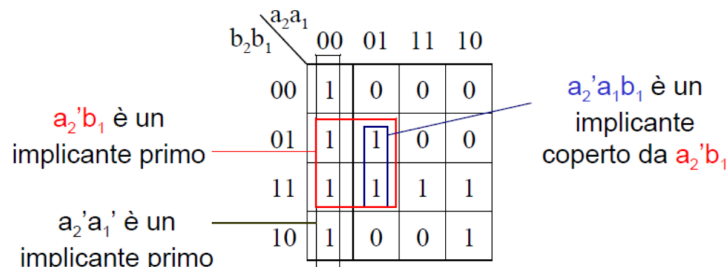


Figura 7: Esempio di implicanti nella mappa di Karnaugh.

5.2.10 Mintermine

La funzione che prende il nome di prodotto fondamentale (o mintermine) è costituita dal prodotto logico dei letterali delle variabili di ingresso, presi negati se nella riga considerata la corrispondente variabile di ingresso vale 0, non negati se tale variabile vale 1.

Si definisce mintermine di una funzione $f(X_1, \dots, X_n)$ un punto B_x nello spazio booleano B_n tale per cui $f(B_x) = 1$.

5.2.11 Maxtermine

La che funzione prende il nome di somma fondamentale (o maxtermine) è costituita dalla somma logica dei letterali delle variabili di ingresso, presi non negati se nella riga considerata la corrispondente variabile di ingresso vale 0, negati se tale variabile vale 1.

Si definisce maxtermine di una funzione $f(X_1, \dots, X_n)$ un punto B_x nello spazio booleano B_n tale per cui $f(B_x) = 0$.

5.2.12 Sintesi canonica

La sintesi canonica non è minima ma può essere minimizzata con diversi algoritmi tra i quali quelli che impiegano la semplificazione algebrica. La sintesi canonica ha però il vantaggio di garantire la massima velocità dato che impiega due livelli di gate S e P o P ed S, ed inoltre permette il riuso degli elementi logici se si devono realizzare più reti logiche con gli stessi ingressi. Non è necessario quindi cercare per forza la sintesi minima.

- **Sintesi canonica SP (Somma di prodotti):** una funzione di n variabili può essere espressa in un solo modo come somma di prodotto di n variabili, ossia di mintermini.
- **Sintesi canonica PS (Prodotto di somme):** una funzione di n variabili può essere espressa in un solo modo come prodotto di somme di n variabili, ossia di maxtermini.

5.3 Multiplexer

Seleziona un singolo input fra diversi segnali in ingresso, in base al valore degli ingressi (dell'ingresso) di selezione. Data la tabella:

A	B	S	Y
a	*	0	a
*	b	1	b

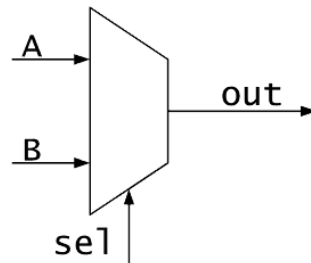


Figura 8: Esempio di un multiplexer a due ingressi.

5.4 Demultiplexer

Il contrario del multiplexer è il demultiplexer che accetta un segnale in serie e lo trasferisce in una delle possibili n uscite a seconda dei $\log(n)$ ingressi di selezione.

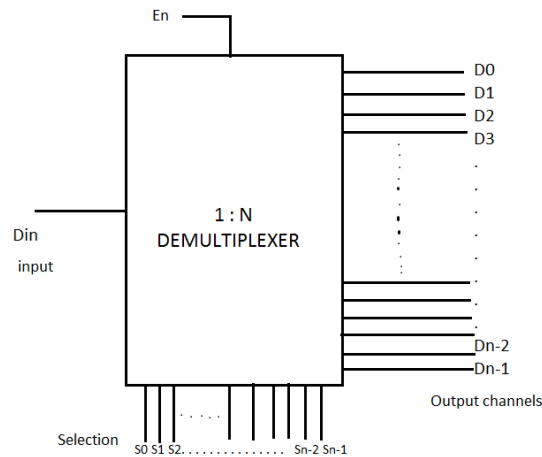


Figura 9: Esempio di un demultiplexer a n uscite e $\log(n)$ ingressi.

5.5 Esercizio struttura calcolatore

Rappresenta con una tabella di verità quattro istruzioni diverse:

1. **mov AL, [64]:** ($AL \rightarrow [64]$), che lavora sulla memoria
2. **mov [64], AL:** ($[64] \rightarrow AL$), che lavora sulla memoria
3. **in AL, [64]:** ($[64] \rightarrow AL$), che lavora su I/O
4. **out [64], AL:** ($AL \rightarrow [64]$), che lavora su I/O

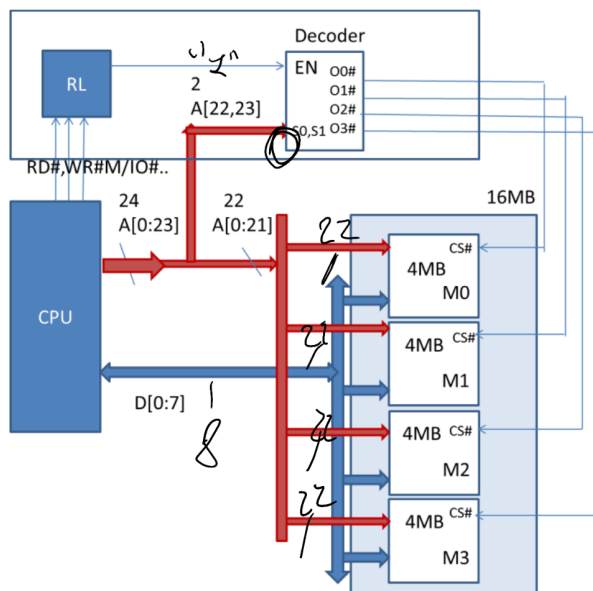


Figura 10: Esempio struttura di un calcolatore.

Tabella di verità:

	RD#	RW#	M-I/O#	M-RD#	M-WR#	I/O-RD#	I/O-WR#
X	0	0	0	-	-	-	-
X	0	0	1	-	-	-	-
2.	0	1	0	1	0	0	0
3.	0	1	1	0	0	1	0
1.	1	0	0	0	1	0	0
4.	1	0	1	0	0	0	1
*	1	1	0	0	0	0	0
*	1	1	1	0	0	0	0

- Le prime due configurazioni sono vietate, perchè la CPU non potrà mai scrivere e leggere contemporaneamente.
- Le quattro configurazioni numerate sono le richieste dell'esercizio.
- Le ultime due configurazioni sono possibili, per esempio nel caso in cui la CPU non debba fare nulla.

5.6 Reti sequenziali

Reti logiche in cui in ogni istante le uscite dipendono non solo dalla configurazione degli ingressi in quell'istante ma anche dalle configurazioni dagli ingressi negli istanti precedenti. Devono, quindi, conservare memoria degli eventi passati nel proprio stato interno, dipendendo così anche dal tempo. Possono essere:

- **Sincrone**, se le variazioni delle configurazioni di ingresso vengono sentite e modificano lo stato e le uscite in ogni istante.
- **Asincrone**, se le variazioni delle configurazioni di ingresso vengono sentite e modificano lo stato e le uscite solo in presenza di un opportuno evento di sincronizzazione.

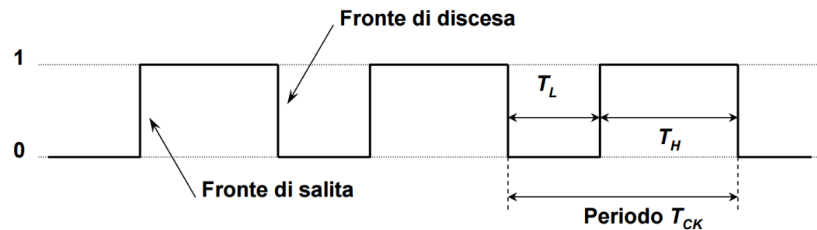


Figura 11: Segnale di Clock.

5.6.1 Alea

Si dice **glitch** un impulso indesiderato dell'uscita in fase transitoria. Se una rete combinatoria può potenzialmente presentare un glitch allora si dice che è caratterizzata da un'alea. In alcuni casi le alea sono eliminabili con una sintesi opportuna, in altri casi non sono eliminabili. Le alea si classificano in:

- **Alea statiche** se, in seguito ad un cambiamento di ingresso, l'uscita nel transitorio ha un cambiamento di valore mentre avrebbe dovuto rimanere costante.
- **Alea dinamiche** se, in seguito ad un cambiamento degli ingressi, l'uscita prima di cambiare valore definitivamente a regime, cambia più volte durante il transitorio.

NB: Le reti canoniche Sp e Ps evitano questo problema.

Si chiama **tempo di propagazione** il tempo impiegato dal gate a propagare in uscita il cambiamento del segnale di ingresso.

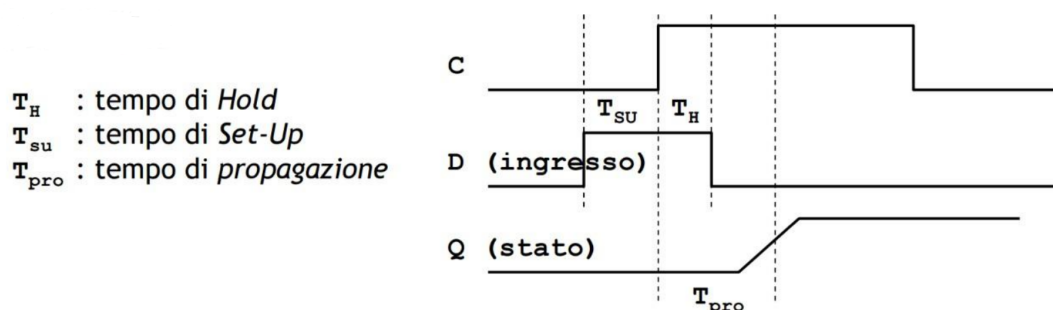


Figura 12: Tempo di Set-Up, Hold e Propagazione.

5.6.2 Elementi di Memeoria

Gli elementi di base delle reti sequenziali sono gli elementi chiamati **bistabili**, capaci di mantenere al loro interno il valore 0 o il valore 1.

5.6.3 Set Reset

Il **Set Reset (SR)** è una rete con due ingressi S e R e una uscita Q (ed una uscita complementata Q#) tale che l'uscita Q assume il valore 1 quando $S = 1$ e $R = 0$ e il valore 0 quando $S = 0$ e $R = 1$ e non cambi allorchè $S = R = 0$. La combinazione di ingresso $S = R = 1$ non si verifica mai.

S	R	Q	Q#
0	0	Q	Q#
0	1	0	1
1	0	1	0
1	1	-	-

5.6.4 Latch e D-Latch

Si definisce **Latch** un bistabile sincrono trasparente, capace di memorizzare o meno segnali di ingresso in funzione di un segnale di abilitazione. Il **D-Latch** è memoria capace di mantenere l'uscita se il segnale di clock non è attivo e di cambiare l'uscita campionando l'ingresso quando il clock è attivo.

C	S	R	Q#	S	R	Q	Q#
0	-	-	Q	-	0	Q	Q#
1	0	0	Q	0	1	0	1
1	0	1	0	0	1	0	1
1	1	0	1	1	1	1	0
1	1	1	-				

5.6.5 Flip Flop

Il **Flip Flop** è un dispositivo bistabile privo della proprietà di trasparenza. Nel flip flop il cambiamento della uscita non è conseguenza del cambiamento dell'ingresso di dato ma è conseguenza del cambiamento di un ingresso di controllo sincrono o asincrono.

5.7 Registri

I FF e i Latch D sono molto usati e se impiegati in parallelo (uno per ogni bit) possono registrare parole di n bit. Formano quindi i registri nella forma più semplice.

5.7.1 Lettura

Se il registro deve essere letto si deve selezionare quale dei registri va verso all'uscita. Nel register file ci sono due uscite possibili. L'uscita è selezionata dal multiplexer che contiene i numeri che corrispondenti ai due registri. Per la lettura l'abilitazione è immediata ma si collega ad un multiplexer che farà passare l'uscita solo della porta abilitata.

5.7.2 Scrittura

Un registro può essere sovrascritto con l'ingresso che arriva su writedata e con il write register che ne indica il numero e lo invia a un decoder che da uscita 1 a solo un registro, solo se c'è anche il segnale di Write che funge da "clock" o abilitatore alla scrittura.

6 ISA (Instruction Set Architecture)

Il vocabolario delle istruzioni di un calcolatore è detto instruction set, possono essere diversi ma comunque sono simili fra loro. I più importanti sono il RISC-V, il MIPS e l'Intel x86.

6.1 RISC-V Instruction Set

Un'architettura aperta sviluppata dall'Università di Berkeley a partire dal 2010 e ora gestito dalla RISC-V foundation.

6.1.1 Operazioni aritmetiche

Ci sono diverse operazioni:

- **Somma aritmetica a tre operandi:** `add x5, x6, x7`.
Nel registro x5 viene messa la somma di x6 e x7.
- **Somma aritmetica immediata:** `addi x5, x6, 20`.
Nel registro x5 viene messa la somma di x6 e 20.
- **Sottrazione aritmetica a tre operandi:** `sub x5, x6, x7`.
Nel registro x5 viene messa la sottrazione di x6 e x7.

6.1.2 Registri nel RISC-V

Il RISC-V ha 32 registri da 32 bit:

- **x0:** contiene il valore 0.
- **x1:** contiene l'indirizzo di ritorno.
- **x2:** puntatore allo stack.
- **x3:** puntatore globale.
- **x4:** puntatore al thread.
- **[x5-x7] e [x28-x31]:** contengono dati temporanei.
- **x8:** puntatore al frame
- **x9 e [x18-x27]:** contengono dati salvati
- **x10 x11:** contengono gli argomenti o i valori di ritorno di una funzione.
- **[x12-x17]:** contengono gli argomenti delle funzioni.

NB: Le operazioni aritmetiche usano operandi a registro.

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5–7	t0–2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10–11	a0–1	Function arguments/return values	Caller
x12–17	a2–7	Function arguments	Caller
x18–27	s2–11	Saved registers	Callee
x28–31	t3–6	Temporaries	Caller

Figura 13: Nomi registri standard RISC-V.

6.1.3 Codifica delle istruzioni

Tutte le istruzioni RISC-V sono codificate come parole a 32-bit. Nel caso delle istruzioni non immediate di tipo R (registry) ha questa struttura:

funct7	rs2	rs1	funct3	rd	opcode
--------	-----	-----	--------	----	--------

Dove:

- **funct7** e **funct3** sono codici operativi aggiuntivi a 7 e 3 bit.
- **rs1** e **rs2** sono i registri del primo e del secondo operando, i registri hanno sempre 5 bit.
- **rd** è il registro di destinazione, sempre 5 bit.
- **opcode** codice operativo dell'istruzione, che ha sempre 7 bit.

Nel caso delle istruzioni tipo I (immediate e load) la struttura è questa:

immediate	rs1	funct3	rd	opcode
-----------	-----	--------	----	--------

Dove il campo immediate è il campo da 12 bit per il valore numerico passato. Nelle istruzioni di tipo S (store) invece è questa:

immediate	rs1	rs2	funct3	immediate	opcode
-----------	-----	-----	--------	-----------	--------

Dove il campo immediate è separato in due campi da 7 e 5 bit per avere i due registri sempre nella stessa posizione.

6.1.4 Salti condizionati

Sono due istruzioni per implementare processi decisionali, simili a un if seguito da go to. Ci sono diversi tipi di operazioni:

- **Branch if equal, BEQ.**
- **Branch if not equal, BNE.**
- **Branch if lower than, BLT.**
- **Branch if greater of equal, BGE.**
- **Branch if less than unsigned, BLTU.**
- **Branch if greater or equal unsigned, BGEU.**

6.1.5 Procedure

Nelle procedure è necessario eseguire diverse operazioni, come il passaggio dei parametri, restituire un risultato, ecc... Nel RISC-V vengono usati diversi registri, dal x10 al x17 per il passaggio degli argomenti e del risultato e x1 per salvare l'indirizzo del chiamante. Esistono alcune istruzioni per chiamare le procedure:

- Istruzione per saltare all'inizio della procedura e salvare l'indirizzo del chiamante in x1: **jal x1, Procedura**
- Istruzione per ritornare al chiamante: **jalr x0, 0(x1)**

6.1.6 Stack e Heap

La struttura dati ideale per copiare il contenuto dei registri è lo **stack**, una coda di tipo LIFO. In RISC-V il puntatore allo stack è il registro x2 (o sp). Lo stack pointer viene incrementato o decrementato di una parola ogni volta che si toglie o si inserisce il contenuto di un registro. Alcuni programmi RISC-V usano il frame pointer (fp, registro x8) per puntare alla prima parola del frame di una procedura, per convenienza rispetto allo sp che invece potrebbe cambiare allocando vettori e strutture locali. Lo stack parte dalla parte alta degli indirizzi della memoria e si espande verso il basso, l'**heap**, l'area riservata agli elementi dinamici si muove al contrario.

7 Architettura della CPU

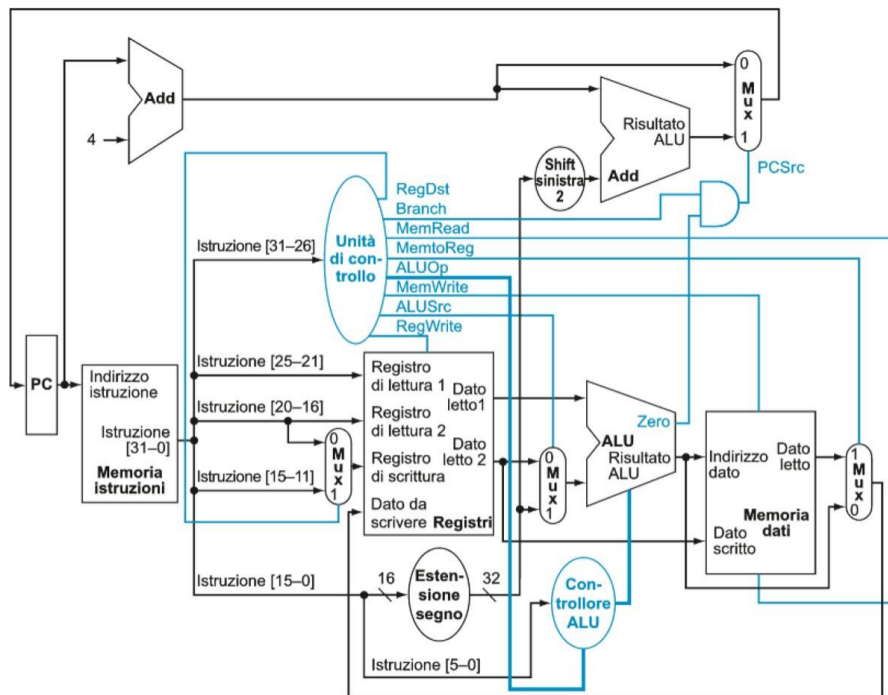


Figura 14: Architettura semplice e monociclo di una CPU Risc-V.

L'esecuzione di un'istruzione si divide in 5 fasi:

- **Fetch:** si occupa di prendere l'istruzione tramite il suo indirizzo nel program counter (PC).
- **Decode:** si occupa di tradurre l'istruzione, dividendo i bit per la fase di execute.
- **Execute:** si occupa di eseguire le varie operazioni, in questa fase i calcoli sono destinati all'alu.
- **Memory:** per alcune istruzioni è necessario attivare la memoria.
- **Write Back:** si occupa di riscrivere il risultato.

L'unità di controllo si occupa di gestire i vari segnali, tra cui:

- **RegWrite:** per scrivere sui registri al termine di un'operazione di ALU o di lettura.
- **MemWrite e MemRead:** per leggere o scrivere i dati sulla memoria di dato.
- **AluOp:** è l'insieme dei segnali di controllo che indicano all'ALU l'operazione da fare.
- **Zero:** il segnale che esce dal PSW della ALU per indicare se c'è stata una condizione di zero.
- **Branch:** segnale che indica se c'è stato un branch.
- **MemToReg:** segnale inviato a un multiplexer che serve per scrivere un dato dalla memoria a un registro.
- **AluSrc:** segnale inviato al multiplexer che si occupa di mandare all'alu il valore di un registro o il valore immediato nell'istruzione.
- **PsScr:** risultato dell'AND fra il segnale zero e il segnale branch, se uguale a uno imposta il pc all'indirizzo del salto.

Altri componenti importanti sono l'estensione del segno da 16 bit a 32 bit e lo shift a sinistra di 2.

7.1 Alee

Esistono molte situazioni in cui la pipeline non è ideale. La pipeline ideale però può nella realtà avere difficoltà nella esecuzione a causa di diversi problemi chiamati alee, con tre tipi di conflitti:

- **Alee strutturali**
- **Alee di dato**
- **Alee di controllo**

7.1.1 Alee strutturali

7.1.2 Alee di dato

7.1.3 Alee di controllo

8 Memorie

La memoria è l'unità logica di memorizzazione di dati nel calcolatore. Nell'architettura dei calcolatori si distingue la memoria centrale, in diretto collegamento con la CPU e che contiene l'informazione in uso, dalla memoria di massa che ha il compito come periferica di I/O di mantenere dati ed istruzioni in modo permanente. Esistono diversi tipi di dispositivi di memoria che possono essere classificati in base a:

- Capacità.
- Caratteristiche fisiche.
- Modalità di accesso.
- Organizzazione.
- Prestazioni.

8.1 Capacità

La capacità è calcolata come:

$$C = M \times N$$

Dove N è la grandezza della parola e M è il numero di parole.

8.2 Caratteristiche fisiche

Le memorie possono essere classificate in base alle caratteristiche fisiche:

- Memorie a semiconduttore.
- Memorie a superficie magnetica.
- Memorie ottiche.

Le memorie centrali sono realizzate con tipologia a semiconduttore; la memoria di massa, quale i dischi, a superficie magnetica o di tipo ottico (come i CD) e sempre di più anche a semiconduttore (memorie a stato solido). Possono essere categorizzate anche in base al consumo, all'affidabilità e in base alla durevolezza. In base alla durevolezza o volatilità, si possono definire due categorie:

- Memorie che perdono le informazioni se non alimentate elettricamente, come le RAM.
- Memorie che mantengono le informazioni anche se non alimentate elettricamente, come l'hard disk o le flash.

Le memorie volatili sono molto utilizzate, anzi costituiscono la massima parte della memoria centrale in quanto hanno tempi di accesso ossia latenza molto inferiori a quelli di altre memorie. Tra le memorie volatili ne esistono diverse categorie:

- **Memorie statiche:** costituite da bistabili sincroni o a sincroni che mantengono il bit memorizzato fino ad una nuova riscrittura con FF o D-Latch.
- **Memorie dinamiche:** il bit è memorizzato sotto forma di carica all'interno di un condensatore, a causa delle correnti di scarica e il valore 1 memorizzato diventa 0. È quindi necessario che periodicamente si provveda a leggere ogni bit e a riscrivere i bit con valore 1.

8.3 Modalità di accesso

L'architettura della memoria definisce la sua modalità di accesso. A seconda della modalità di accesso si hanno diverse prestazioni e diversi tempi di accesso:

- **Accesso sequenziale:** se per accedere ad un dato in una fissata posizione della memoria è necessario accedere a tutti i dati memorizzati in posizioni precedenti.
- **Accesso diretto:** se è possibile avere un accesso direttamente alla locazione di memoria voluta senza dovere accedere alle locazioni precedenti.
- **Accesso casuale:** è un accesso diretto in cui il tempo di accesso è costante e non dipende né dall'indirizzo né dalla precedente locazione acceduta.
- **Accesso associativo:** e per accedere ad un dato bisogna cercare un'associazione tra l'indirizzo e il contenuto.

8.3.1 Memorie RAM

Le memorie RAM (Random access memory) sono memorie volatili a lettura e scrittura in cui il tempo di accesso è costante per ogni locazione di memoria indirizzata. Perciò hanno per definizione modalità di accesso casuale. Ci sono due categorie:

- **SRAM (Static RAM).**
- **DRAM (Dynamic RAM).**

8.3.2 Memorie ROM

Le memorie ROM (Read only memory) sono memorie di sola lettura, esse non hanno logica dedicate per la scrittura ma vengono scritte attraverso differenti apparati tecnologici:

- **PROM (Programmable ROM):** programmabili una sola volta impiegando un pin speciale.
- **EPROM (Eraseable programmable ROM):** PROM ma cancellabili in toto con luce ultravioletta.
- **EEPROM (Electrically eraseable programmable ROM):** PROM che sono cancellabili elettricamente senza doverle estrarre dalla scheda.
- **FLASH:** sono un tipo speciale di EEPROM leggibili e scrivibili a blocchi senza estrarle dal circuito.

8.4 Organizzazione della memoria

La memoria vista fino ad ora e' una tipica architettura non gerarchica dove tutte le memorie sono uguali dal punto di vista logico nello spazio di indirizzamento. Chi progetta il calcolatore vede tutti i dispositivi di memoria centrale nello stesso modo anche se hanno tipologie diverse.

8.4.1 Organizzazione gerarchica

Il calcolatore contiene memorie di tipi, prestazioni e costi diverse. La gerarchia e' fatta in modo tale da avere ai livelli piu' alti memorie piu' piccole e veloci, ma con un meccanismo di accesso che ottimizza l'uso di tutte le memorie. Questo è possibile grazie a due proprietà:

- **Località spaziale:** è la proprietà dei programmi per cui accedendo ad un dato è assai probabile che si debba accedere ad un dato allocato in uno spazio in memoria localmente vicino.
- **Località temporale:** è la proprietà dei programmi per cui accedendo ad un dato è assai probabile che si debba accedere nuovamente ad esso in un breve intervallo di tempo.

I programmi non vedono la gerarchia ma referenziano i dati come se fossero sempre in memoria centrale, perciò bisogna mantenere il più vicino possibile alla CPU dati utilizzati più recentemente.

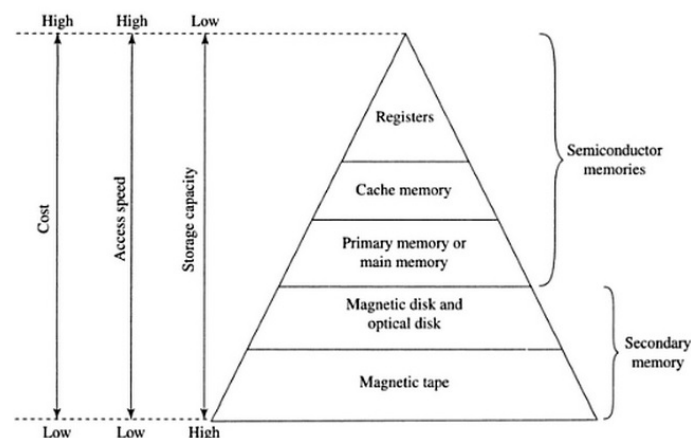


Figura 15: Piramide della gerarchia delle memorie.

8.4.2 Memorie Cache

La cache è una memoria centrale veloce, volatile e ad accesso associativo. Le memorie cache sono realizzate normalmente con tecnologia SRAM.

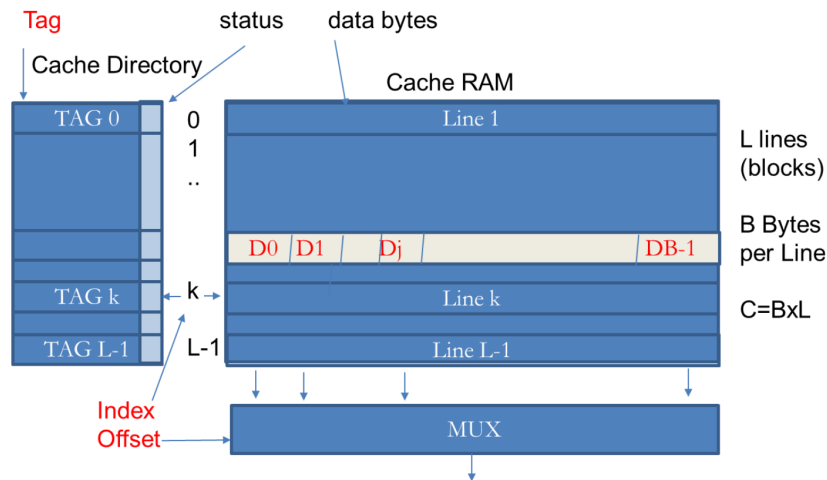


Figura 16: Descrizione grafica cache.

Nello specifico:

- La cache RAM contiene L blocchi di dati o linee, ogni linea contiene B byte. La capacità è quindi $C = L \times B$.
- Può essere calcolato un offset nell'indirizzo partendo da B , nello specifico l'offset è $\log_2(B)$.
- L'offset serve a il multiplexer per controllare se c'è stata una hit.
- Essendo un accesso associativo, non è detto che il dato si trovi in cache. Esiste perciò una parte della cache detta anche cache directory che contiene i tag ed eventuali bit di stato.
- I bit di tag sono tutti quelli rimanenti tolto l'offset, quindi $\text{tag} = n_a - \log_2(B)$. Dove n_a è il numero di bit dell'indirizzo.

La cache può essere classificata a seconda del grado di associatività per quel che riguarda il piazzamento dei dati, in tre categorie:

- **Cache fully associative:** è la cache descritta precedentemente con piazzamento completamente associativo ossia senza vincoli sull'indice in cui la linea può essere piazzata. L'indirizzo è quindi composto da una parte di offset e una parte di Tag che vengono usati per essere associati a tutte le linee della cache. Questa architettura è la più efficiente, ha mediamente la miss rate più bassa ma è anche la più costosa per il costo della logica di controllo dei tag. È inoltre la più costosa anche per decidere la politica di rimpiazzamento nel caso di cache piena.
- **Cache direct mapped:** la cache direct mapped prevede una sola possibilità di piazzamento per ogni linea. Ogni linea infatti usa una parte del tag come chiave unica di indice. Ogni linea se è presente sulla cache può risiedere in una sola posizione data dalla parte di indirizzo chiamata index. La parte di index è la parte meno significativa del tag. In questo modo due blocchi adiacenti vengono mappati in due linee consecutive e non si sovrappongono. L'indirizzo è quindi composto da una parte di offset, da una parte di index di $N_i = \log_2(L)$, restanti sono i bit di Tag che vengono usati per essere associati al tag memorizzato all'interno. È la cache potenzialmente con il più alto miss rate ma è anche la meno costosa. Di fatto l'index fa da indirizzo per indirizzare solo quella linea sia della cache directory che della cache ram che può contenere il dato.

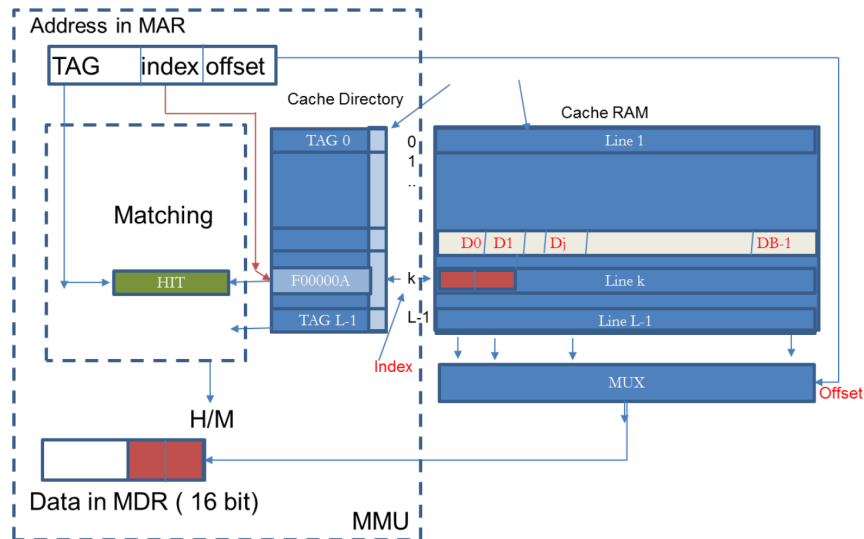


Figura 17: Descrizione grafica cache.

- **Cache n-way associative** è la cache più usata, compromesso tra le architetture precedenti. Le linee della cache sono divise in S set da n vie ognuna. Ogni linea del livello inferiore della gerarchia può essere piazzata in un solo set ma in una qualsiasi delle n linee dentro al set. È una cache quindi direct mapped a livello di set ma totalmente associativa dentro al set. Il miss rate è minore rispetto al caso precedente perché ci sono più possibilità di mantenere la linea nella cache senza doverla rimpiazzare. L'indirizzo è composto da una parte di offset, da una parte di index di $N_i = \log_2(S)$ dove $S = \frac{L}{n}$ è il numero di set e i restanti bit di Tag che vengono usati per essere associati al tag memorizzato all'interno.

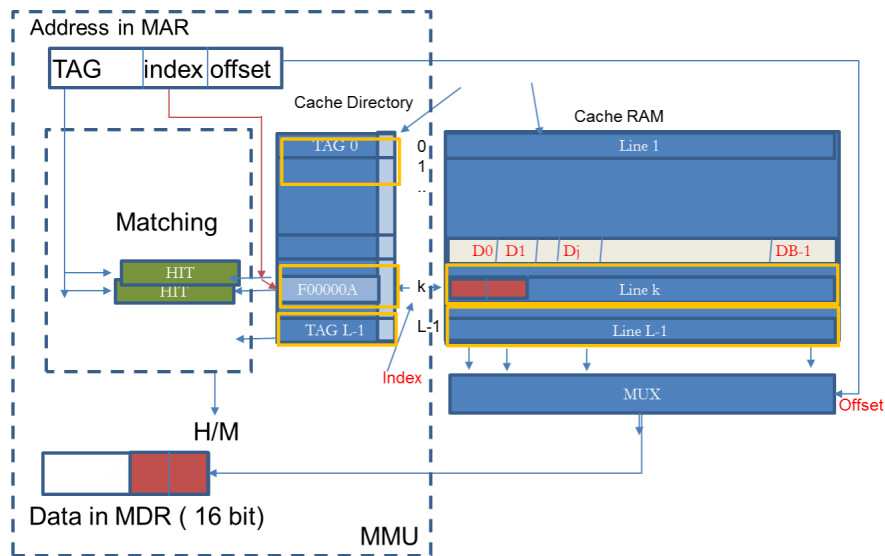


Figura 18: Descrizione grafica cache n-way associativa.

8.4.3 Prestazioni nell'organizzazione gerarchica

Le prestazioni delle memorie si misurano in tempo, la latenza per le memorie si chiama tempo di accesso mentre il throughput si calcola come reciproco del tempo di ciclo. Nello specifico:

- **Blocco:** unità di informazione minima scambiata fra livelli.
- **Hit:** un accesso alla memoria che trova l'informazione cercata nel livello superiore.
- **Hit rate:** frequenza di accessi trovati nel livello superiore (h).

- **Miss rate:** $1-h$.
- **Hit time:** tempo di accesso al livello superiore T_h .
- **Miss penalty:** tempo per rimpiazzare un blocco nel livello superiore più il tempo per fornirlo alla CPU. Quindi $T_{mp} = T_h + T_{miss}$.

Il tempo di accesso può essere calcolato come:

$$T_{acc} = hT_h + (1-h)T_{mp} = T_h + (1-h)T_{miss}$$

8.5 Miss

Ci sono tre tipologie di miss diverse:

- **Compulsory (obbligatorie)** se sono inevitabili, esse si verificano quando la linea richiesta non è mai stata in cache.
- **Conflict** se sono dovute a un conflitto dovute alla limitata associatività, ovvero, il dato viene richiesto nella cache, sarebbe stato presente ma è stato rimpiazzato perché un'altra linea con lo stesso index è stata richiesta in seguito e a causa della limitata associatività ha preso il suo posto.
- **Capacity** se sono dovute alla limitata capacità della cache, ovvero, la linea sarebbe stata presente ma essendo la cache di capacità limitata essa è stata rimpiazzata da un'altra.

8.5.1 Esercizio:

Si deve eseguire un programma con un ciclo di 10 iterazioni. In ogni ciclo si devono leggere le variabili var1, var2, var3 e var4 che sono allocate in memoria agli indirizzi rispettivamente F000A023, F000A021, A000A022, F000A025.

1. Con uno spazio di indirizzamento a 32 bit ed una cache direct mapped con linee di 16B di dati e di capacità di 1Mbyte, quante miss e di che tipo si verificano?
2. E se la cache è a 2 vie associative?
3. E se la cache è come nel primo caso ma con 32 B di dimensione di Linea?
4. E se è come il secondo e terzo caso insieme?

Nel caso 1. si ha che la cache ha 256k righe, quindi ci sono 10 bit di tag, 18 bit di indice e 4 bit di offset.

	Tag	Index	Offset	
Var1	1111000000	000000101000000010	0011	Miss compulsory
Var2	1111000000	000000101000000010	0001	Hit
Var3	1010000000	000000101000000010	0010	Miss compulsory E rimpiazza la precedente stesso index
Var4	1111000000	000000101000000010	0011	Miss conflict (sarebbe stata in cache se non fosse stata rimpiazzata da var3)

Figura 19: Descrizione grafica cache.

Le quattro variabili hanno lo stesso index ma tag diverso, quindi, nella prima iterazione:

- Var1 ha un miss compulsory, perchè la cache è vuota.
- Var2 ha un hit, perchè avendo lo stesso tag di var1 sono state caricate pure var2 e var4.
- Var3 ha un miss compulsory, perchè non è presente nella cache e quindi viene pure caricata.
- Var4 ha un miss conflict, perchè era stata caricata ma poi è stata sostituita da var3, però viene caricata in cache.

In tutte le altre iterazioni le prime due variabili fanno una hit, dato che var4 al fine di ogni iterazione viene caricato, invece var3 e var 4 ha in miss conflict, perchè si sostituiscono a vicenda.

8.6 Scrittura

Avendo più livelli di memorie cache ci sono diverse strategie per scrivere i dati:

- **write-back:** è la strategia per cui la scrittura avviene solo al livello più alto della gerarchia e viene copiato nei livelli più bassi solo se la linea viene rimpiazzata o al termine della esecuzione del task. È il metodo più efficiente dal punto di vista del tempo di accesso.
- **write through:** è la strategia per cui ad ogni scrittura il dato viene scritto in tutti i livelli della gerarchia.

8.7 Memoria virtuale

Memoria virtuale è il nome che viene dato al livello della gerarchia delle memorie che si occupa del trasferimento dei dati tra la memoria principale e il disco.

La memoria virtuale permette ai singoli programmi di espandere il proprio spazio di indirizzamento oltre i limiti della memoria principale. Caratteristica ancora più importante è il fatto che la memoria virtuale supporta la condivisione della memoria principale tra più processi attivi simultaneamente, in modo protetto. La memoria viene divisa in blocchi (chiamate PAGINE) ognuna con propri bit di stato, tra cui quelli P di protezione per indicare che solo alcuni owner possono averne l'accesso.

Il meccanismo della memoria virtuale vede la memoria centrale come una cache della memoria di massa, dove i blocchi sono le pagine. La memoria centrale è fully associative e il meccanismo di rimpiazzamento è LRU.

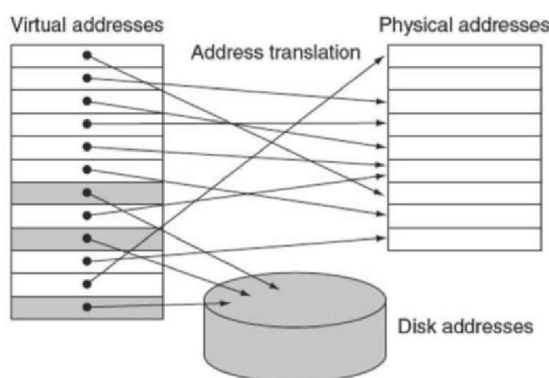


Figura 20: Descrizione grafica memoria virtuale.

La parte alta non funziona come tag con un accesso associativo ma viene usato come indice in un processo che è chiamato address translation ed avviene attraverso una tabella (page table).

8.7.1 Page fault

Il page fault richiede l'accesso alla memoria di massa che può essere 100 000 volte più lento e il processo di gestione è fatto in software non in hardware con milioni di cicli di clock, così si possono usare algoritmi intelligenti per la allocazione. La memoria è completamente associativa per avere il minor numero di miss ossia di page fault. La tecnica di scrittura è sempre write-back, il write through sarebbe troppo costoso. La pagina è a sua volta in memoria, ad un indirizzo indicato in un registro specifico, ossia il PTR Page table register.

8.7.2 Address mapping

In base alla traduzione dell'indirizzo, alcune pagine si trovano effettivamente in memoria centrale, ossia in memoria fisica e la traduzione trasforma il virtual page number in un physical page number. Il primo accesso si ottiene la rilocalizzazione dell'indirizzo virtuale in fisico e l'allocazione di memoria.

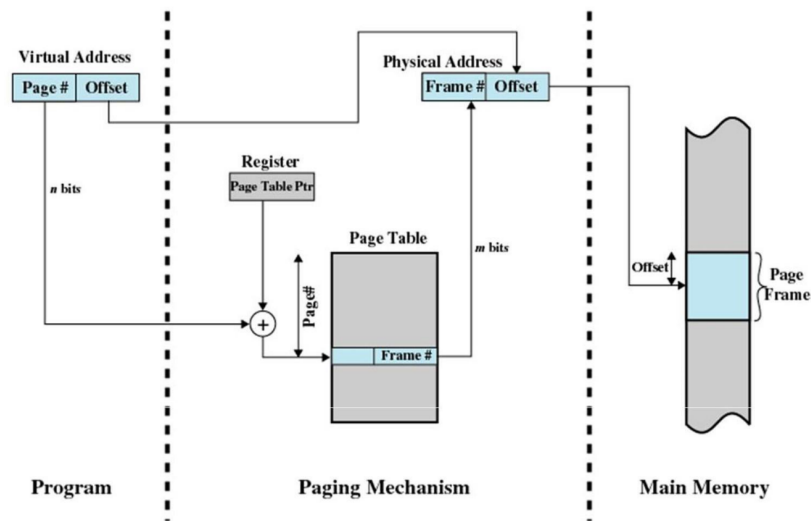


Figura 21: Descrizione grafica memoria virtuale.

8.7.3 TLB (Translation Lookaside Buffer)

Per velocizzare il multiplo accesso in memoria si usa una cache di indirizzi che si chiama Translation Lookaside Buffer (TLB), una piccola cache che contiene le entries della page table usate recentemente ed è direttamente nel processore.

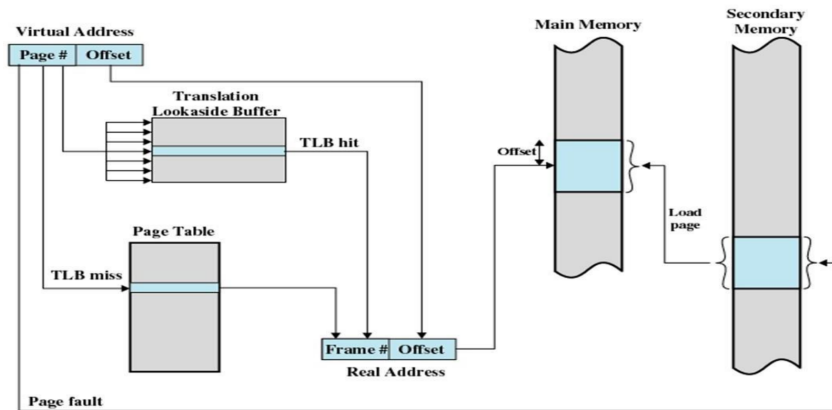


Figura 22: Descrizione grafica memoria virtuale.

9 Codici di Controllo

Per il controllo degli errori si usano dei bit di ridondanza della parola, per esempio una parola che viene memorizzata sul registro o sulla memoria inizialmente a m bit diventa una parola di codice di n bit con $n = m + r$ essendo r i bit di ridondanza.

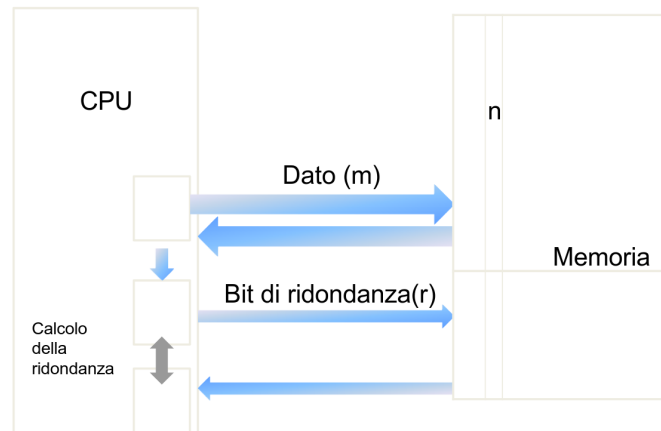


Figura 23: Esempio di bit di ridondanza nelle RAM ECC.

9.1 Distanza di Hamming

La distanza di Hamming tra due variabili binarie di n bit vale d (numero intero tra 0 e n) se d è il numero di cifre binarie diverse tra le due variabili. Si calcola come:

$$\text{EXOR: } d = \sum_{i=0}^{n-1} h_i$$

a_i	a'_i	h_i
0	0	0
0	1	1
1	0	1
1	1	0

NB:

- Se si vogliono rilevare k errori singoli è necessaria una parola con distanza $d=k+1$, perché con un codice di distanza $k+1$ non esiste modo che k errori singoli trasformino una parola valida in una altra parola valida.
- Se si vogliono correggere k errori singoli è necessaria una parola con distanza $d=2k+1$, perché in questo modo pur cambiando k bit la parola in codice sarà comunque più vicina alla parola originaria che a tutte le altre e quindi si può risalire alla parola originaria.

9.2 Codice di parità

Il codice più semplice è il bit di parità (pari) che aggiunge 1 bit per fare in modo che il numero degli 1 nella parola sia un numero pari. Ha una distanza $d=2$ e quindi rileva solo 1 solo errore.

ESEMPIO: Progettare una rete che indichi se due ingressi binari sono entrambi uguali a zero, se il segnale di parità pari è corretto, altrimenti indichi errore.

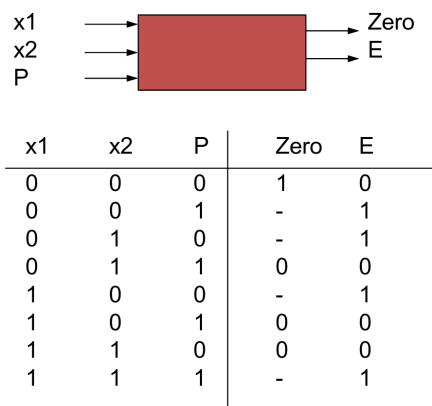


Figura 24: Esempio di codice di parità.

9.3 Codice di Hamming

Il codice di Hamming fornisce un algoritmo per generare codici ridondanti correttivi, tali che sia palese l'indicazione degli eventuali bit errati nella parola codice. Per ognuna delle 2^m parole legali codificate con n bit è necessario che il numero di parole sia:

$$2^n \geq 2^m(n+1)$$

Così per ognuna delle 2^m parole legali che usano n bit si ha 1 parola giusta e n parole sbagliate ma vicine solo a quella legale:

$$2^r \geq m + r + 1$$

9.3.1 Esempio: Esercizio con $m=4$

Supponiamo di avere $m = 4$ bit, che sia **1011**.

1. Trovare i valori della parola di n bit, dove $n = m + r$, con r che sono i bit di ridondanza. Dalla disequazione precedente sappiamo che per $m = 4$ abbiamo $r = 3$ e $n = 7$.
2. Mettere i bit nella posizione corretta, nello specifico i bit di ridondanza dovranno essere messi in posizioni che sono potenze del 2. **NB:** Si parte da 1 e non da 0.

r_1	r_2	m_1	r_3	m_2	m_3	m_4
b_1	b_2	b_3	b_4	b_5	b_6	b_7

Le m_i sono i valori della parola originale, gli r_i invece vanno calcolati.

3. Per calcolare i valori r_i è necessario creare due matrici:

$$H = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ \cancel{x} & \emptyset & 1 & \emptyset & 1 & 0 & 1 \\ \emptyset & \cancel{x} & 1 & \emptyset & 0 & 1 & 1 \\ \emptyset & \emptyset & 0 & \cancel{x} & 1 & 1 & 1 \end{pmatrix}$$

H è una matrice $r \times n$ in cui vengono scritti in binario posizionati verticalmente i valori da 1 a n. E poi vengono cancellate le colonne corrispondenti agli r_i per creare la matrice G.

$$G = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

G, invece, è la matrice creata a partire da H con le colonne cancellate. Nello specifico, per ogni colonna in ordine, se è cancellata vanno ricopiati i bit della riga che sono rimasti, se non è cancellata va copiata una riga della matrice identità.

4. Per codificare il messaggio è necessario moltiplicare la matrice G per il messaggio m e fare modulo 2:

$$G \cdot m = \begin{pmatrix} 2 \\ 3 \\ 1 \\ 2 \\ 0 \\ 1 \\ 1 \end{pmatrix} \text{ modulo } 2 = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}$$

Quindi il messaggio codificato N è: 0110011

5. Per controllarlo è sufficiente moltiplicare il messaggio codificato N per la matrice H.

$$m = H \cdot N = \begin{pmatrix} 1 \\ 3 \\ 2 \end{pmatrix} \text{ modulo } 2 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$$

Quindi il bit trasmesso errato è il numero 011, ossia il numero 3.

10 Parallelismo

Un'architettura parallela è un insieme di elementi di elaborazione che cooperano e comunicano per risolvere velocemente problemi di dimensioni considerevoli, talvolta intrattabili su macchine sequenziali. A livello di problema e di parallelismo computazionale si può definire il parallelismo:

- **Disponibile (available parallelism)** nel programma o ancor meglio nell'algoritmo.
- **Utilizzato (utilized parallelism)** ottenuto durante l'esecuzione.

Compito dei calcolatori è cercare di rendere efficiente lo sfruttamento di tutto il parallelismo disponibile in una applicazione, a due livelli:

- **Parallelismo funzionale:** operazioni e parti delle soluzioni che possono essere eseguite nello stesso tempo.
- **Parallelismo di dato:** strutture dati e parti di esse su cui il programma può lavorare indipendentemente e nello stesso tempo.

Esiste una altra classificazione, che riguarda l'aspetto architetturale e che divide il parallelismo in due categorie:

- **Parallelismo Temporale (pipelining):** Un insieme di unità funzionali sono utilizzate in sequenza per realizzare una singola computazione.
- **Parallelismo Spaziale (Replicazione o parallelismo):** un insieme di unità funzionali sono replicate e utilizzate nello stesso tempo per realizzare una singola o diverse computazioni.

Una architettura parallela risolve due problemi:

- **Speed-up:** lo stesso problema viene risolto in minor tempo.

- **Scale-up:** risolve un problema più grande nello stesso tempo.

Scalabilità Forte (strong scaling) misura l'incremento di velocità, mantenendo fissa la dimensione del problema ed aumentando hardware.

Scalabilità debole (weak scaling) l'incremento di velocità quando le dimensioni del problema crescono proporzionalmente alla crescita dell'hardware.

Ci sono 4 categorie di architetture:

1. **SISD: Single Instruction Stream, Single Data Stream** singoli processori tradizionali con una unità di elaborazione e una unità di controllo.
2. **MISD: Multiple Instruction Stream, Single Data Stream** sono le architetture in pipeline in cui un flusso di dati viene elaborato in cascata da istruzioni diverse in unità diverse.
3. **SIMD: Single Instruction Stream, Multiple Data Stream** architetture con molte unità di elaborazione parallele che elaborano dati diversi con lo stesso flusso di istruzioni, gli array e i processori vettoriali.
4. **MIMD: Multiple Instruction Stream, Multiple Data Stream** architetture composte da più unità di elaborazione più o meno lascamente connesse, ossia tutte le macchine parallele e distribuite attuali e le architetture multicore.

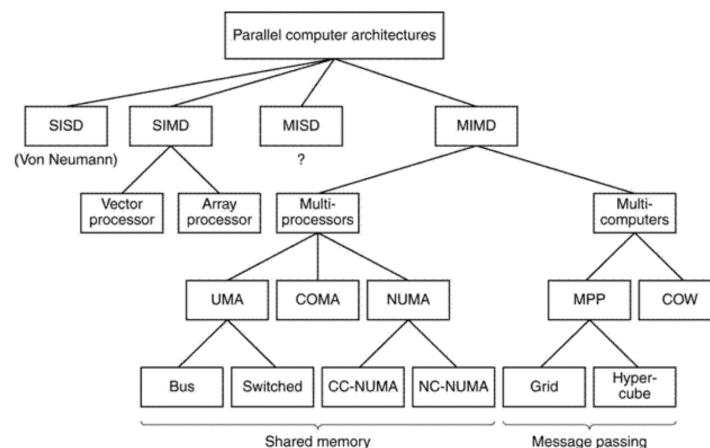


Figura 25: Tabelle delle architetture.

10.1 Architetture MIMD

L'architettura MIMD ha portato allo sviluppo di tutti i server paralleli, i cluster multi-lama e anche all'interno del chip alla realizzazione delle architetture Multicore, Many core.

- **Multiprocessori (shared memory multiprocessors o SMP):** Replicazione di processori e di moduli di memoria condivisa con un global address space. Sono anche detti Shared - memory MIMD architectures perché lo spazio di memoria è condiviso tra tutti i processori e comunicano con variabili condivise e con meccanismi di sincronizzazione attraverso i diversi livelli di memoria. L'architettura è nata in genere per pochi processori e ora è la architettura di riferimento nei sistemi multicore e many core.
- **Multi-computer (distributed memory architecture):** Replicazione di PE (processing elements), come coppia processore/memoria, anche dette Distributed-memory MIMD architectures perché la memoria è distribuita e privata di ogni processore o message-passing MIMD architectures perché la comunicazione avviene con scambi di messaggi. I processori che lavorano in message passing come architettura parallela spesso usano un modello che si chiama SPMD Single Program Multiple Data, in cui di fatto i diversi processori eseguono lo stesso programma su dati diversi, ma permettendo un flusso di istruzioni diverso.

10.1.1 UMA e NUMA

Le architetture multi-processor possono avere gli accessi alla memoria secondo due metodologie:

- **UMA (Unified Memory Access):** Ogni processore ha lo stesso tempo di accesso alla memoria.
- **NUMA (Non Unified Memory Access):** Alcuni accessi sono più o meno veloci a seconda di quale processore richiede l'accesso.

10.2 Architettura Multiple-Issue

Architettura studiata per avere un $CPI < 1$ (oppure un $IPC > 1$), è necessario far iniziare più istruzioni in un ciclo.

10.3 Architettura Superpipeline

Architettura in cui in ogni stadio molte operazioni sono eseguite in porzioni del clock differenti. in questo modo più istruzioni possono partire assieme (nello stesso clock) ottenendo quindi un $CPI < 1$.

10.4 Architettura VLIW (Very Long Instruction Word)

Architettura che almeno in alcune operazioni, esegue un numero fissato di istruzioni >1 , con più unità funzionali in parallelo. Tali istruzioni sono formattate in una sola istruzione in modo staticamente determinato dai compilatori. Questa è una scelta definita della ISA e realizzato dal compilatore al tempo di compilazione.

10.5 Architettura Superscalare

Architettura progettata per eseguire un numero variabile di istruzioni contemporaneamente in unità funzionali diverse, decise in modo dinamico. La scelta di quali istruzioni eseguire dipende dall'unità di instruction fetch che:

1. Acquisisce più istruzioni assieme..
2. Verifica se alcune istruzioni possono essere eseguite su unità funzionali diverse e disponibili.

L'impiego efficiente è ottimizzato dai compilatori che mettono vicine istruzioni che non hanno dipendenze o anti-dipendenze. È perciò una scelta di microarchitettura.

10.6 Esecuzione Speculativa

Lo scopo primario è di non porre in stallo la pipeline cercando di evitare le alee RAW e le alee WAW e WAR in caso di multiple pipeline. **Si dice Esecuzione speculativa se l'esecuzione di un blocco avviene prima di sapere se il blocco deve essere eseguito.** È necessario il supporto sia del compilatore che di hardware dedicato. I risultati vanno in registri temporanei, gli scratch register, che poi vengono copiati sui registri veri solo se il blocco doveva davvero essere eseguito. Esiste una tabella che tiene traccia di che blocco è eseguito o è in esecuzione.

10.6.1 RAT (Register Alias Table)

Avviene quindi internamente una rinomina dei registri in scratch register che è necessaria sia per non modificare i registri veri e sia per evitare alee di tipo WAW e WAR quando più unità lavorano in parallelo e in più pipeline.

10.6.2 Fuori Ordine

La soluzione seguente permettere una esecuzione fuori ordine, dove ogni blocco può essere eseguito anche non nell'esatto ordine deciso dal programmatore e dal compilatore, con i soli vincoli che:

- I dati vanno prelevati in ordine (in-order issue).
- I dati vanno riscritti in ordine alla fine (in-order commit).

10.7 Interrupt e Polling

La gestione degli eventi e della comunicazione con I/O può creare delle interruzioni e può essere gestita in due modi:

- **Polling:** Il metodo piu' semplice, a controllo di programma con interrogazione periodica. La CPU verifica lo stato della periferica, continua la richiesta fino a che non verifica che e' pronta per ricevere dati in uscita o fornire dati in ingresso e poi eventualmente inizia il trasferimento dati. Se necessario, ricomincia.
- **Interrupt:** Sospensione forzata del processo di esecuzione e trasferimento di controllo ad una routine di servizio (ISR) che soddisfa le richieste dell'evento che ha provocato l'interruzione, al termine della quale il controllo viene restituito al processo sospeso.

10.7.1 Interruzioni

Le interruzioni possono essere di due tipi:

- **Sincrone:** sono dette anche eccezioni e sono dovute al programma e sono di due tipi:
 - **Processor-detected exception.**
 - **Programmed exceptions.**
- **Asincrone:** sono di tipo hardware e derivano da sorgenti esterne come da periferiche di I/O e non sono relative all'istruzioni che si stanno eseguendo.

10.7.2 Faults

Le faults sono istruzioni che non si devono eseguire, come ad esempio eseguire una istruzione senza avere i privilegi, o scrivere in una parte read only. Sono riscontrate prima di incrementare il program counter e bloccano il programma. L'istruzione successiva non viene eseguita.

10.7.3 Traps

Le traps sono dovute disolito al debugger e sono una chiamata ad un programam che è la funzione del debugger, sono eseguite dopo aver incrementato il PC e poi il programma va avanti. Le trap anomalie di funzionamento rilevate e gestite immediatamente dopo dell'esecuzione dell'istruzione che le genera.

10.7.4 Aborts

Le aborts anomalie di funzionamento che non permettono di individuare esattamente l'istruzione che le hanno determinate: ad esempio un malfunzionamento segnalato da una periferica.